Day 11 Task Workflow — Friday, Mar 7, 2026

> **Sprint:** 3 (ML Pipeline → Intelligence Integration) | **Phase:** Real-Time Inference + Alert Engine
> **Owner:** Lead Architect | **Status:** ⬤ Ready to Start
> **Goal:** Connect ML models to live traffic, auto-generate alerts, begin LLM Gateway
> **Grade:** Week 3 Day 1 A COMPLETE ☑ | Week 3 Day 2 STARTING ⬤

Day 10 Results Context (Critical)

Three-model ensemble is TRAINED and DEPLOYED on VPS:

```
Ensemble — Acc: 80.66% | P: 92.50% | R: 71.85% | F1: 80.87% | AUC: 0.9312

Individual Models:
  IF  — Acc: 79.68% | P: 97.26% | R: 66.16% | F1: 78.75% | AUC: 0.9378
  RF  — Acc: 74.16% | F1w: 69.45%                          | AUC: 0.9576
  AE  — Acc: 60.39% | P: 87.07% | R: 35.72% | F1: 50.66% | AUC: 0.8517

Saved Models:
  /app/ml/saved_models/isolation_forest.pkl     (1.4 MB)
  /app/ml/saved_models/random_forest.pkl        (29.9 MB)
  /app/ml/saved_models/autoencoder/model.keras  (205 KB)
  /app/ml/saved_models/autoencoder/threshold.npy (144 B)

ML API: /api/v1/ml/models ☑ | /api/v1/ml/comparison ☑
Training time: 98 seconds on VPS (4 vCPU, TF 2.18.0)
```

Key: AUC-ROC scores are strong (0.85-0.96) — models discriminate well. Performance gap is documented and expected for NSL-KDD test set.

Day 11 Objective

Connect the trained ML models to the live capture pipeline so that by end of day:

- ML Worker subscribes to `flows:live` Redis channel, scores every flow with all 3 models + ensemble
- Scored flows have `is_anomaly` and `anomaly_score` updated in PostgreSQL
- Alert Engine auto-creates alerts when ensemble composite score ≥ 0.30
- Alerts published to `alerts:live` Redis channel → WebSocket → browser
- Feature preprocessing pipeline converts live flow features → model-ready input
- LLM Gateway service scaffolded with multi-provider routing
- ML Worker Docker container stops restarting and runs stably

> **NOTE:** Frontend tasks remain with Full-Stack Dev. This covers **Lead Architect tasks only.**

Scope Adherence Checklist

Requirement	Source Document	Section
ML Worker inference pipeline	MASTER_DOC_PART4	§8.1-8.3
Real-time scoring latency < 200ms	MASTER_DOC_PART4	§8.2
Alert auto-creation from anomalies	MASTER_DOC_PART2	§4.2 (alerts table)
Alert severity from composite score	MASTER_DOC_PART4	§1.2
Redis pub/sub alerts:live channel	MASTER_DOC_PART2	§5.2 (WebSocket events)
LLM Gateway multi-provider routing	MASTER_DOC_PART4	§9.1
LLM prompt templates	MASTER_DOC_PART4	§9.2
ML API: POST /ml/predict	MASTER_DOC_PART2	§5.1

Architectural Constraints

Constraint	Rationale	Enforcement
Redis pub/sub for ML→Alert pipeline	PART2 §6.1	Not HTTP polling
Async scoring (non-blocking)	PART4 §8.2	asyncio in worker
Pre-loaded models in memory	PART4 §8.3	ModelManager at startup
StandardScaler for live features	PART4 §2.4	Same scaler as training
Alert categories from RF labels	PART4 §5.3	dos, probe, r2l, u2r
LLM provider routing by task type	PART4 §9.1	DeepSeek=analysis, Groq=realtime, GLM=bulk
Streaming responses for chat	PART4 §9.3	SSE via FastAPI

Task Breakdown

TASK 1 — Feature Preprocessing Pipeline 🟤

Time Est: 60 min | **Priority:** 🟤 Critical

Live capture flows have 63 features (40 NSL-KDD + 23 extended). The ML models were trained on 40 StandardScaler-transformed features. We need a preprocessing pipeline that converts live flow features → model input.

1.1 Create `ml/inference/preprocessor.py`

```
"""
ThreatMatrix AI — Live Flow Preprocessor

Converts raw flow feature dicts (from capture engine) into model-ready
numpy arrays using the same preprocessing as training.
```

```
Pipeline:
1. Extract 40 NSL-KDD features from flow dict
2. Encode categoricals (protocol_type, service, flag) with stored LabelEncoders
3. Scale numericals with stored StandardScaler
4. Return numpy array ready for model inference
"""

from __future__ import annotations

import logging
from pathlib import Path
from typing import Any, Dict, List, Optional

import joblib
import numpy as np

logger = logging.getLogger(__name__)

# The 40 NSL-KDD feature names in training order
NSL_KDD_FEATURES = [
    "duration", "protocol_type", "service", "flag",
    "src_bytes", "dst_bytes", "land", "wrong_fragment", "urgent",
    "hot", "num_failed_logins", "logged_in", "num_compromised",
    "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "is_host_login", "is_guest_login",
    "count", "srv_count", "serror_rate", "srv_serror_rate",
    "rerror_rate", "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate",
    "dst_host_count", "dst_host_srv_count", "dst_host_same_srv_rate",
    "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
    "dst_host_srv_diff_host_rate", "dst_host_serror_rate",
    "dst_host_srv_serror_rate", "dst_host_rerror_rate",
    "dst_host_srv_rerror_rate",
]

CATEGORICAL_FEATURES = ["protocol_type", "service", "flag"]

SAVE_DIR = Path(__file__).parent.parent / "saved_models"


class FlowPreprocessor:
    """
    Preprocess live flow features for ML inference.

    Uses the LabelEncoders and StandardScaler fitted during training
    (saved by NSLKDDLoader.preprocess with fit=True).
    """

    def __init__(self, models_dir: Optional[Path] = None) -> None:
        self.models_dir = models_dir or SAVE_DIR
        self.label_encoders: Dict = {}
        self.scaler = None
        self._loaded = False
```

```
    def load(self) -> None:
        """Load fitted encoders and scaler from training artifacts."""
        encoder_path = self.models_dir / "preprocessor_encoders.pkl"
        scaler_path = self.models_dir / "preprocessor_scaler.pkl"

        if encoder_path.exists() and scaler_path.exists():
            self.label_encoders = joblib.load(encoder_path)
            self.scaler = joblib.load(scaler_path)
            self._loaded = True
            logger.info("[Preprocessor] Loaded encoders + scaler from %s",
self.models_dir)
        else:
            logger.warning(
                "[Preprocessor] Preprocessor artifacts not found. "
                "Run training first to generate encoders/scaler."
            )

    def preprocess_flow(self, flow_features: Dict[str, Any]) ->
Optional[np.ndarray]:
        """
        Convert a single flow feature dict to model-ready numpy array.

        Args:
            flow_features: Dict from FeatureExtractor.extract() (63 features)

        Returns:
            1D numpy array of shape (40,) or None if preprocessing fails.
        """
        if not self._loaded:
            logger.error("[Preprocessor] Not loaded. Call load() first.")
            return None

        try:
            # Extract the 40 NSL-KDD features in order
            values = []
            for feat_name in NSL_KDD_FEATURES:
                val = flow_features.get(feat_name, 0)

                if feat_name in CATEGORICAL_FEATURES:
                    # Encode categorical using stored LabelEncoder
                    le = self.label_encoders.get(feat_name)
                    if le is not None:
                        val_str = str(val)
                        if val_str in le.classes_:
                            val = le.transform([val_str])[0]
                        else:
                            val = -1  # Unknown category
                    else:
                        val = 0
                elif isinstance(val, bool):
                    val = int(val)
                else:
                    val = float(val) if val is not None else 0.0
```

```
                values.append(val)

            # Convert to numpy and scale
            X = np.array(values, dtype=np.float32).reshape(1, -1)
            X_scaled = self.scaler.transform(X)

            return X_scaled[0]  # Return 1D array

        except Exception as exc:
            logger.error("[Preprocessor] Failed to preprocess flow: %s", exc)
            return None

    def preprocess_batch(self, flows: List[Dict[str, Any]]) ->
Optional[np.ndarray]:
        """
        Preprocess a batch of flows.

        Returns:
            2D numpy array of shape (n_flows, 40) or None.
        """
        results = []
        for flow in flows:
            processed = self.preprocess_flow(flow)
            if processed is not None:
                results.append(processed)

        if not results:
            return None

        return np.array(results, dtype=np.float32)
```

1.2 Save Preprocessing Artifacts During Training

Update `train_all.py` to save the encoders and scaler after preprocessing:

```
# After loader.preprocess(train_df, fit=True):
import joblib

# Save preprocessing artifacts for live inference
joblib.dump(loader.label_encoders, SAVE_DIR / "preprocessor_encoders.pkl")
joblib.dump(loader.scaler, SAVE_DIR / "preprocessor_scaler.pkl")
logger.info("Preprocessing artifacts saved for live inference")
```

Verification:

#	Check	Expected
1	Import works	`from ml.inference.preprocessor import FlowPreprocessor`

#	Check	Expected
2	Encoders file saved after training	`preprocessor_encoders.pkl` exists
3	Scaler file saved after training	`preprocessor_scaler.pkl` exists
4	Preprocess a mock flow dict	Returns 1D array shape (40,)
5	All 40 features present	No missing features
6	Categorical encoding works	"tcp" → integer
7	Unknown categories handled	Returns -1
8	Batch preprocessing	Returns (N, 40) array
9	Scaled values	Mean ≈ 0, Std ≈ 1

TASK 2 — ML Worker (Real-Time Inference) 🟥

Time Est: 120 min | **Priority:** 🟥 Critical

Implement `ml/inference/worker.py` — the Redis subscriber that scores every live flow.

Per MASTER_DOC_PART4 §8.1:

```
flows:live → ML Worker → [score] → PostgreSQL (update) + alerts:live (publish)
```

2.1 Implement `ml/inference/worker.py`

```python
"""
ThreatMatrix AI — ML Inference Worker

Per MASTER_DOC_PART4 §8: Real-time inference pipeline.

Subscribes to 'flows:live' Redis channel.
For each flow:
  1. Deserialize flow JSON
  2. Preprocess features (40 NSL-KDD features, scaled)
  3. Run through all 3 models
  4. Ensemble scoring → composite score + severity
  5. Update flow in PostgreSQL (anomaly_score, is_anomaly)
  6. If anomalous: create alert + publish to alerts:live
"""

from __future__ import annotations

import asyncio
import json
import logging
```

```python
import time
from typing import Any, Dict, Optional

import redis.asyncio as redis

from ml.inference.model_manager import ModelManager
from ml.inference.preprocessor import FlowPreprocessor

logger = logging.getLogger(__name__)


class MLWorker:
    """
    Real-time ML inference worker.

    Subscribes to Redis flows:live channel and scores
    every flow using the three-model ensemble.
    """

    def __init__(
        self,
        redis_url: str = "redis://redis:6379",
        flow_channel: str = "flows:live",
        alert_channel: str = "alerts:live",
        db_url: Optional[str] = None,
    ) -> None:
        self.redis_url = redis_url
        self.flow_channel = flow_channel
        self.alert_channel = alert_channel
        self.db_url = db_url

        self.model_manager = ModelManager()
        self.preprocessor = FlowPreprocessor()
        self._redis: Optional[redis.Redis] = None
        self._publisher: Optional[redis.Redis] = None
        self._running = False

        # Stats
        self.stats = {
            "flows_scored": 0,
            "anomalies_detected": 0,
            "alerts_created": 0,
            "errors": 0,
            "start_time": 0.0,
            "avg_inference_ms": 0.0,
        }

    async def start(self) -> None:
        """Initialize and start the worker."""
        logger.info("[Worker] Starting ML inference worker...")

        # Load models
        status = self.model_manager.load_all()
        if not any(status.values()):
```

```
            logger.error("[Worker] No ML models available. Cannot start
inference.")
            raise RuntimeError("No ML models loaded")

        logger.info("[Worker] Models loaded: %s", status)

        # Load preprocessor
        self.preprocessor.load()
        if not self.preprocessor._loaded:
            logger.error("[Worker] Preprocessor not loaded. Run training first.")
            raise RuntimeError("Preprocessor artifacts not found")

        # Connect to Redis
        self._redis = redis.from_url(
            self.redis_url, decode_responses=True,
            socket_connect_timeout=5, socket_keepalive=True,
        )
        self._publisher = redis.from_url(
            self.redis_url, decode_responses=True,
        )

        await self._redis.ping()
        logger.info("[Worker] Connected to Redis at %s", self.redis_url)

        self._running = True
        self.stats["start_time"] = time.time()

        # Subscribe and process
        pubsub = self._redis.pubsub()
        await pubsub.subscribe(self.flow_channel)
        logger.info("[Worker] Subscribed to channel: %s", self.flow_channel)

        try:
            async for message in pubsub.listen():
                if not self._running:
                    break
                if message["type"] != "message":
                    continue

                try:
                    await self._process_message(message["data"])
                except Exception as exc:
                    self.stats["errors"] += 1
                    logger.error("[Worker] Error processing flow: %s", exc)

        except asyncio.CancelledError:
            logger.info("[Worker] Cancelled — shutting down")
        finally:
            await pubsub.unsubscribe(self.flow_channel)
            await self._cleanup()

    async def _process_message(self, data: str) -> None:
        """Process a single flow message from Redis."""
        start = time.time()
```

```
        # Parse flow
        msg = json.loads(data)
        if msg.get("event") != "new_flow":
            return

        flow_data = msg.get("payload", {})
        features = flow_data.get("features", {})
        flow_id = flow_data.get("id")

        if not features:
            return

        # Preprocess
        import numpy as np
        X = self.preprocessor.preprocess_flow(features)
        if X is None:
            return

        X_batch = X.reshape(1, -1)

        # Score with ensemble
        results = self.model_manager.score_flows(X_batch)
        if not results:
            return

        result = results[0]
        composite_score = result["composite_score"]
        is_anomaly = result["is_anomaly"]
        severity = result["severity"]

        self.stats["flows_scored"] += 1

        # Update flow in PostgreSQL (via Redis message for backend to consume)
        score_update = {
            "event": "flow_scored",
            "payload": {
                "flow_id": flow_id,
                "anomaly_score": composite_score,
                "is_anomaly": is_anomaly,
                "severity": severity,
                "label": result["label"],
                "model_agreement": result["model_agreement"],
                "if_score": result["if_score"],
                "ae_score": result["ae_score"],
                "rf_label": result["rf_label"],
                "rf_confidence": result["rf_confidence"],
            },
        }

        await self._publisher.publish("ml:scored", json.dumps(score_update))

        # If anomalous, create alert
        if is_anomaly and severity in ("critical", "high", "medium"):
```

```
            self.stats["anomalies_detected"] += 1
            await self._create_alert(flow_data, result)

        # Stats
        elapsed_ms = (time.time() - start) * 1000
        self.stats["avg_inference_ms"] = (
            (self.stats["avg_inference_ms"] * (self.stats["flows_scored"] - 1) +
elapsed_ms)
            / self.stats["flows_scored"]
        )

        if self.stats["flows_scored"] % 100 == 0:
            logger.info(
                "[Worker] Stats: %d scored | %d anomalies | %d alerts | %.1fms
avg",
                self.stats["flows_scored"],
                self.stats["anomalies_detected"],
                self.stats["alerts_created"],
                self.stats["avg_inference_ms"],
            )

    async def _create_alert(self, flow_data: Dict, result: Dict) -> None:
        """Create and publish an alert for an anomalous flow."""
        # Map RF label → alert category per PART4 §5.3
        category_map = {
            "dos": "ddos",
            "probe": "port_scan",
            "r2l": "unauthorized_access",
            "u2r": "privilege_escalation",
        }

        alert = {
            "event": "new_alert",
            "payload": {
                "severity": result["severity"],
                "category": category_map.get(result["label"], "anomaly"),
                "title": f"{result['severity'].upper()} — {result['label']}
detected",
                "description": (
                    f"ML ensemble detected {result['label']} activity. "
                    f"Composite score: {result['composite_score']:.2f}. "
                    f"Model agreement: {result['model_agreement']}."
                ),
                "source_ip": flow_data.get("src_ip", "unknown"),
                "dest_ip": flow_data.get("dst_ip", "unknown"),
                "composite_score": result["composite_score"],
                "model_agreement": result["model_agreement"],
                "rf_label": result["rf_label"],
                "rf_confidence": result["rf_confidence"],
                "if_score": result["if_score"],
                "ae_score": result["ae_score"],
                "flow_id": flow_data.get("id"),
            },
        }
```

```
        await self._publisher.publish(self.alert_channel, json.dumps(alert))
        self.stats["alerts_created"] += 1

        logger.info(
            "[Worker] ALERT: %s — %s (score=%.2f, agreement=%s)",
            result["severity"].upper(),
            result["label"],
            result["composite_score"],
            result["model_agreement"],
        )

    async def _cleanup(self) -> None:
        """Clean up connections."""
        self._running = False
        if self._redis:
            await self._redis.close()
        if self._publisher:
            await self._publisher.close()
        logger.info("[Worker] Shut down. Final stats: %s", self.stats)

    def get_stats(self) -> Dict[str, Any]:
        """Return current worker statistics."""
        stats = self.stats.copy()
        if stats["start_time"] > 0:
            stats["uptime_seconds"] = time.time() - stats["start_time"]
        return stats


async def main() -> None:
    """Entry point for standalone worker execution."""
    import os

    redis_url = os.environ.get("REDIS_URL", "redis://redis:6379")
    worker = MLWorker(redis_url=redis_url)
    await worker.start()


if __name__ == "__main__":
    asyncio.run(main())
```

2.2 Update ML Worker Dockerfile/Command

The `ml-worker` service in `docker-compose.yml` currently restarts because there's no working entry point. Update to:

```
  ml-worker:
    build: ./backend
    container_name: tm-ml-worker
    command: python -m ml.inference.worker
    environment:
```

```
      - REDIS_URL=redis://redis:6379
      -
DATABASE_URL=postgresql+asyncpg://threatmatrix:${DB_PASSWORD}@postgres:5432/threat
matrix
    depends_on:
      redis:
        condition: service_healthy
      postgres:
        condition: service_healthy
    restart: unless-stopped
```

Verification:

#	Check	Expected
1	Worker starts without crashing	"Subscribed to flows:live" log
2	Models loaded at startup	All 3 show "loaded"
3	Preprocessor loaded	"Loaded encoders + scaler"
4	Processes flow messages	"flows_scored" incrementing
5	Inference latency < 200ms	avg_inference_ms < 200
6	Anomalies detected	"anomalies_detected" > 0 after some flows
7	Alerts published	Messages on alerts:live channel
8	Stats log every 100 flows	Periodic log messages
9	Graceful shutdown	Clean shutdown on SIGTERM
10	ml-worker container stops restarting	`docker compose ps` shows "Up"

TASK 3 — Alert Engine 🟤

Time Est: 90 min | **Priority:** 🟤 Critical

Create a service that consumes ML-scored alerts from Redis and persists them to the `alerts` table in PostgreSQL.

3.1 Create `backend/app/services/alert_engine.py`

```
"""
ThreatMatrix AI — Alert Engine

Subscribes to 'alerts:live' Redis channel.
Persists ML-generated alerts to PostgreSQL alerts table.
Broadcasts to WebSocket for real-time browser updates.

Per MASTER_DOC_PART2 §4.2 (alerts table schema).
"""
```

```python
from __future__ import annotations

import asyncio
import json
import logging
import uuid
from datetime import datetime, timezone
from typing import Any, Dict, Optional

import redis.asyncio as redis
from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine

logger = logging.getLogger(__name__)


class AlertEngine:
    """
    Consumes anomaly alerts from ML Worker and persists to PostgreSQL.
    """

    def __init__(
        self,
        redis_url: str = "redis://redis:6379",
        database_url: str = "",
        alert_channel: str = "alerts:live",
    ) -> None:
        self.redis_url = redis_url
        self.database_url = database_url
        self.alert_channel = alert_channel
        self._redis: Optional[redis.Redis] = None
        self._engine = None
        self._running = False
        self.stats = {"alerts_persisted": 0, "errors": 0}

    async def start(self, db_engine=None) -> None:
        """Start the alert engine."""
        logger.info("[AlertEngine] Starting...")

        if db_engine:
            self._engine = db_engine
        elif self.database_url:
            from sqlalchemy.ext.asyncio import create_async_engine
            self._engine = create_async_engine(self.database_url)

        self._redis = redis.from_url(
            self.redis_url, decode_responses=True,
            socket_connect_timeout=5,
        )
        await self._redis.ping()

        self._running = True
        pubsub = self._redis.pubsub()
```

```
        await pubsub.subscribe(self.alert_channel)
        logger.info("[AlertEngine] Subscribed to %s", self.alert_channel)

        try:
            async for message in pubsub.listen():
                if not self._running:
                    break
                if message["type"] != "message":
                    continue
                try:
                    await self._process_alert(message["data"])
                except Exception as exc:
                    self.stats["errors"] += 1
                    logger.error("[AlertEngine] Error: %s", exc)
        except asyncio.CancelledError:
            pass
        finally:
            await pubsub.unsubscribe(self.alert_channel)
            if self._redis:
                await self._redis.close()

    async def _process_alert(self, data: str) -> None:
        """Process and persist an alert."""
        msg = json.loads(data)
        if msg.get("event") != "new_alert":
            return

        payload = msg["payload"]

        if self._engine is None:
            logger.warning("[AlertEngine] No DB engine — alert not persisted")
            return

        # Insert alert per MASTER_DOC_PART2 §4.2 schema
        async with AsyncSession(self._engine) as session:
            alert_id = str(uuid.uuid4())
            now = datetime.now(timezone.utc)

            insert_sql = text("""
                INSERT INTO alerts (
                    id, severity, category, title, description,
                    source_ip, destination_ip, confidence,
                    status, ml_scores, flow_ids, created_at, updated_at
                ) VALUES (
                    :id, :severity, :category, :title, :description,
                    :source_ip, :dest_ip, :confidence,
                    'new', :ml_scores, :flow_ids, :created_at, :updated_at
                )
            """)

            ml_scores = json.dumps({
                "composite": payload.get("composite_score"),
                "isolation_forest": payload.get("if_score"),
                "autoencoder": payload.get("ae_score"),
```

```
                "random_forest_label": payload.get("rf_label"),
                "random_forest_confidence": payload.get("rf_confidence"),
                "model_agreement": payload.get("model_agreement"),
            })

            flow_ids = f'{{{payload.get("flow_id", "")}}}' if
payload.get("flow_id") else '{}'

            await session.execute(insert_sql, {
                "id": alert_id,
                "severity": payload.get("severity", "medium"),
                "category": payload.get("category", "anomaly"),
                "title": payload.get("title", "ML Anomaly Detected"),
                "description": payload.get("description", ""),
                "source_ip": payload.get("source_ip"),
                "dest_ip": payload.get("dest_ip"),
                "confidence": payload.get("composite_score", 0.0),
                "ml_scores": ml_scores,
                "flow_ids": flow_ids,
                "created_at": now,
                "updated_at": now,
            })
            await session.commit()

        self.stats["alerts_persisted"] += 1
        logger.info(
            "[AlertEngine] Alert persisted: %s — %s (score=%.2f)",
            payload.get("severity"),
            payload.get("category"),
            payload.get("composite_score", 0.0),
        )
```

3.2 Integrate AlertEngine into FastAPI Lifespan

Add to `backend/app/main.py` lifespan:

```
# In lifespan context manager, after FlowConsumer:
from app.services.alert_engine import AlertEngine

alert_engine = AlertEngine(
    redis_url=settings.redis_url,
    database_url=settings.database_url,
)
alert_task = asyncio.create_task(alert_engine.start(db_engine=engine))
```

Verification:

#	Check	Expected
1	AlertEngine starts	"Subscribed to alerts:live"

#	Check	Expected
2	Receives ML alerts	Processes messages from worker
3	Inserts into PostgreSQL	New rows in alerts table
4	Alert has correct schema	severity, category, source_ip, ml_scores
5	ml_scores JSONB stored	composite, if, ae, rf scores
6	flow_ids array stored	Reference to source flow
7	Stats tracking	alerts_persisted incrementing
8	Query alerts	`SELECT * FROM alerts ORDER BY created_at DESC LIMIT 5`

TASK 4 — Flow Score Updater 🟢

Time Est: 45 min | **Priority:** 🟢 Medium

Create a service that consumes `ml:scored` messages and updates the `is_anomaly` + `anomaly_score` columns in the `network_flows` table.

4.1 Create `backend/app/services/flow_scorer.py`

```
"""
ThreatMatrix AI — Flow Score Updater

Subscribes to 'ml:scored' Redis channel.
Updates network_flows with anomaly_score and is_anomaly.
"""

from __future__ import annotations

import asyncio
import json
import logging
from typing import Optional

import redis.asyncio as redis
from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession

logger = logging.getLogger(__name__)


class FlowScoreUpdater:
    """Updates flow records with ML scoring results."""

    def __init__(
        self,
        redis_url: str = "redis://redis:6379",
        channel: str = "ml:scored",
```

```
    ) -> None:
        self.redis_url = redis_url
        self.channel = channel
        self._redis: Optional[redis.Redis] = None
        self._engine = None
        self._running = False
        self.stats = {"flows_updated": 0, "errors": 0}

    async def start(self, db_engine=None) -> None:
        """Start the score updater."""
        self._engine = db_engine
        self._redis = redis.from_url(self.redis_url, decode_responses=True)
        await self._redis.ping()

        self._running = True
        pubsub = self._redis.pubsub()
        await pubsub.subscribe(self.channel)
        logger.info("[FlowScorer] Subscribed to %s", self.channel)

        try:
            async for message in pubsub.listen():
                if not self._running:
                    break
                if message["type"] != "message":
                    continue
                try:
                    await self._process_score(message["data"])
                except Exception as exc:
                    self.stats["errors"] += 1
                    logger.error("[FlowScorer] Error: %s", exc)
        except asyncio.CancelledError:
            pass
        finally:
            await pubsub.unsubscribe(self.channel)
            if self._redis:
                await self._redis.close()

    async def _process_score(self, data: str) -> None:
        """Update flow with ML score."""
        msg = json.loads(data)
        if msg.get("event") != "flow_scored":
            return

        payload = msg["payload"]
        flow_id = payload.get("flow_id")
        if not flow_id or not self._engine:
            return

        async with AsyncSession(self._engine) as session:
            update_sql = text("""
                UPDATE network_flows
                SET anomaly_score = :score,
                    is_anomaly = :is_anomaly
                WHERE id = :flow_id
```

```
            """)
            await session.execute(update_sql, {
                "score": payload.get("anomaly_score", 0.0),
                "is_anomaly": payload.get("is_anomaly", False),
                "flow_id": flow_id,
            })
            await session.commit()

        self.stats["flows_updated"] += 1
```

Verification:

#	Check	Expected
1	Subscribes to ml:scored	"Subscribed" log
2	Updates flow records	is_anomaly, anomaly_score set
3	Query scored flows	`SELECT anomaly_score FROM network_flows WHERE is_anomaly = true`

TASK 5 — LLM Gateway Scaffolding 🟢

Time Est: 90 min | **Priority:** 🟢 Medium

Begin the LLM Gateway per MASTER_DOC_PART4 §9.1. This is Week 4 deliverable but scaffolding starts now.

5.1 Create `backend/app/services/llm_gateway.py`

```
"""
ThreatMatrix AI — LLM Gateway Service

Per MASTER_DOC_PART4 §9.1: Multi-provider LLM routing.

Providers:
  - DeepSeek V3: Complex analysis, reasoning ($0.14/M in)
  - Groq Llama 3.3 70B: Real-time alerts, fast queries ($0.06/M)
  - GLM-4-Flash: Bulk tasks, translations ($0.01/M)

Routing:
  - Complex analysis → DeepSeek
  - Real-time alerts → Groq
  - Bulk/translation → GLM
  - Fallback → next available
"""

from __future__ import annotations

import logging
import os
```

```
from enum import Enum
from typing import Any, AsyncIterator, Dict, List, Optional

logger = logging.getLogger(__name__)


class LLMProvider(str, Enum):
    """Available LLM providers."""
    DEEPSEEK = "deepseek"
    GROQ = "groq"
    GLM = "glm"


class TaskType(str, Enum):
    """Task types for provider routing."""
    ALERT_ANALYSIS = "alert_analysis"
    DAILY_BRIEFING = "daily_briefing"
    IP_INVESTIGATION = "ip_investigation"
    CHAT = "chat"
    TRANSLATION = "translation"
    QUICK_SUMMARY = "quick_summary"


# Provider routing per PART4 §9.1
TASK_ROUTING: Dict[TaskType, List[LLMProvider]] = {
    TaskType.ALERT_ANALYSIS: [LLMProvider.DEEPSEEK, LLMProvider.GROQ],
    TaskType.DAILY_BRIEFING: [LLMProvider.DEEPSEEK, LLMProvider.GLM],
    TaskType.IP_INVESTIGATION: [LLMProvider.DEEPSEEK, LLMProvider.GROQ],
    TaskType.CHAT: [LLMProvider.DEEPSEEK, LLMProvider.GROQ],
    TaskType.TRANSLATION: [LLMProvider.GLM, LLMProvider.DEEPSEEK],
    TaskType.QUICK_SUMMARY: [LLMProvider.GROQ, LLMProvider.GLM],
}

# Prompt templates per PART4 §9.2
PROMPTS = {
    "alert_analysis": """You are ThreatMatrix AI Analyst, an expert cybersecurity
analyst.
Analyze the following network security alert and provide:
1. A clear explanation of what happened
2. Why this is dangerous
3. Recommended immediate actions
4. Long-term remediation steps

Alert Details:
- Severity: {severity}
- Category: {category}
- Source IP: {source_ip} → Destination IP: {dest_ip}
- ML Confidence: {confidence:.0%}
- Model Agreement: {model_agreement}

ML Scores:
- Isolation Forest: {if_score:.2f}
- Random Forest: {rf_label} ({rf_confidence:.0%})
- Autoencoder: {ae_score:.2f}
```

```
- Composite: {composite_score:.2f}

Provide your analysis in a clear, professional format.""",

    "daily_briefing": """You are ThreatMatrix AI, generating a daily cyber threat
briefing.

Network Statistics (Last 24 Hours):
- Total flows analyzed: {total_flows}
- Anomalous flows detected: {anomaly_count} ({anomaly_pct:.1f}%)
- Alerts generated: {alert_count}
  - Critical: {critical}, High: {high}, Medium: {medium}, Low: {low}
- Top attack categories: {top_categories}

Current Threat Level: {threat_level}

Generate a concise executive briefing covering:
1. Overall threat posture assessment
2. Key incidents and patterns
3. Recommendations for the security team
4. Predicted risk trend for next 24 hours""",

    "ip_investigation": """Investigate the following IP address for potential
threats:

IP: {ip_address}
Internal observations:
- Flows involving this IP: {flow_count}
- Anomalous flows: {anomaly_count}
- Protocols used: {protocols}
- Ports accessed: {ports}
- First seen: {first_seen}
- Last seen: {last_seen}

Provide a risk assessment with confidence level.""",

    "translation": """Translate the following cybersecurity alert/report to
Amharic (አማርኛ).
Maintain technical terms in English where Amharic equivalents don't exist.
Keep the professional tone.

Text to translate:
{text}""",
}


class LLMGateway:
    """
    Multi-provider LLM gateway with fallback routing.
    """

    def __init__(self) -> None:
        self.providers: Dict[LLMProvider, Dict[str, Any]] = {}
        self.stats = {
```

```
            "requests": 0,
            "tokens_in": 0,
            "tokens_out": 0,
            "cost_usd": 0.0,
        }
        self._init_providers()

    def _init_providers(self) -> None:
        """Initialize available providers from environment."""
        deepseek_key = os.environ.get("DEEPSEEK_API_KEY")
        groq_key = os.environ.get("GROQ_API_KEY")
        glm_key = os.environ.get("GLM_API_KEY")

        if deepseek_key:
            self.providers[LLMProvider.DEEPSEEK] = {
                "api_key": deepseek_key,
                "base_url": "https://api.deepseek.com/v1",
                "model": "deepseek-chat",
                "cost_per_m_in": 0.14,
                "cost_per_m_out": 0.28,
            }
            logger.info("[LLM] DeepSeek provider initialized")

        if groq_key:
            self.providers[LLMProvider.GROQ] = {
                "api_key": groq_key,
                "base_url": "https://api.groq.com/openai/v1",
                "model": "llama-3.3-70b-versatile",
                "cost_per_m_in": 0.06,
                "cost_per_m_out": 0.06,
            }
            logger.info("[LLM] Groq provider initialized")

        if glm_key:
            self.providers[LLMProvider.GLM] = {
                "api_key": glm_key,
                "base_url": "https://open.bigmodel.cn/api/paas/v4",
                "model": "glm-4-flash",
                "cost_per_m_in": 0.01,
                "cost_per_m_out": 0.01,
            }
            logger.info("[LLM] GLM provider initialized")

        if not self.providers:
            logger.warning("[LLM] No API keys configured. LLM features disabled.")

    def get_prompt(self, task_type: str, **kwargs: Any) -> str:
        """Build a prompt from template."""
        template = PROMPTS.get(task_type, "")
        try:
            return template.format(**kwargs)
        except KeyError as e:
            logger.error("[LLM] Missing prompt variable: %s", e)
            return template
```

```
    def select_provider(self, task_type: TaskType) -> Optional[LLMProvider]:
        """Select the best available provider for the task."""
        routing = TASK_ROUTING.get(task_type, list(LLMProvider))
        for provider in routing:
            if provider in self.providers:
                return provider
        return None

    def get_status(self) -> Dict[str, Any]:
        """Return gateway status."""
        return {
            "providers": {
                p.value: {"available": p in self.providers}
                for p in LLMProvider
            },
            "stats": self.stats,
            "prompts_available": list(PROMPTS.keys()),
        }

    # Full chat/streaming implementation will be completed in Week 4
    # when API keys are configured and LLM endpoints are built
```

Verification:

#	Check	Expected
1	Import works	`from app.services.llm_gateway import LLMGateway`
2	Prompt templates load	4 templates available
3	Task routing defined	6 task types mapped to providers
4	Provider init (no keys)	Warning logged, gateway disabled
5	get_prompt formats	Template with kwargs filled
6	get_status returns info	providers, stats, prompts_available

---

TASK 6 — ML Predict Endpoint 🟢

Time Est: 45 min | **Priority:** 🟢 Medium

Add `POST /ml/predict` endpoint per MASTER_DOC_PART2 §5.1.

6.1 Add to `backend/app/api/v1/ml.py`

```
from pydantic import BaseModel
from typing import Any, Dict, List

class PredictRequest(BaseModel):
    """Request body for ML prediction."""
```

```
    features: Dict[str, Any]  # Flow feature dict

class PredictResponse(BaseModel):
    """Response from ML prediction."""
    composite_score: float
    severity: str
    is_anomaly: bool
    label: str
    model_agreement: str
    scores: Dict[str, float]


@router.post("/predict", response_model=PredictResponse)
async def predict_flow(request: PredictRequest):
    """Score a flow with the ML ensemble."""
    from ml.inference.preprocessor import FlowPreprocessor
    from ml.inference.model_manager import ModelManager

    preprocessor = FlowPreprocessor()
    preprocessor.load()

    manager = ModelManager()
    manager.load_all()

    X = preprocessor.preprocess_flow(request.features)
    if X is None:
        raise HTTPException(status_code=400, detail="Failed to preprocess
features")

    import numpy as np
    results = manager.score_flows(X.reshape(1, -1))
    if not results:
        raise HTTPException(status_code=500, detail="Scoring failed")

    r = results[0]
    return PredictResponse(
        composite_score=r["composite_score"],
        severity=r["severity"],
        is_anomaly=r["is_anomaly"],
        label=r["label"],
        model_agreement=r["model_agreement"],
        scores={
            "isolation_forest": r["if_score"],
            "autoencoder": r["ae_score"],
            "random_forest_confidence": r["rf_confidence"],
        },
    )
```

Verification:

#	Check	Expected

#	Check	Expected
1	Endpoint in /docs	OpenAPI shows POST /ml/predict
2	POST with flow features	Returns composite score + severity
3	Normal flow features	severity = "none", is_anomaly = false
4	Anomalous features	severity = "medium"+ , is_anomaly = true

---

Files Created/Modified Today

```
threatmatrix-ai/
├── backend/
│   ├── ml/
│   │   ├── inference/
│   │   │   ├── preprocessor.py          🆕 TASK 1
│   │   │   └── worker.py               🔨 TASK 2 (full implementation)
│   │   ├── training/
│   │   │   └── train_all.py            🔨 TASK 1 (save preprocessor artifacts)
│   │   └── saved_models/
│   │       ├── preprocessor_encoders.pkl  🆕 TASK 1 (after retrain)
│   │       └── preprocessor_scaler.pkl    🆕 TASK 1 (after retrain)
│   ├── app/
│   │   ├── services/
│   │   │   ├── alert_engine.py          🆕 TASK 3
│   │   │   ├── flow_scorer.py           🆕 TASK 4
│   │   │   └── llm_gateway.py           🆕 TASK 5
│   │   ├── api/v1/
│   │   │   └── ml.py                   🔨 TASK 6 (add predict endpoint)
│   │   └── main.py                     🔨 TASK 3 (alert engine lifespan)
│   └── docker-compose.yml             🔨 TASK 2 (ml-worker command)
└── docs/worklog/
    └── DAY_11_MAR07.md                 🆕 This file
```

Verification Checklist

> **CRITICAL: ML inference pipeline touches every layer. Numerical errors propagate silently into alerts.**

#	Verification	Expected
1	Preprocessor imports	No errors
2	Preprocessor loads encoders + scaler	Files exist after retrain
3	Mock flow → 40-feature array	Shape (40,) with scaled values

#	Verification	Expected
4	Unknown service/protocol handled	Returns -1 (no crash)
5	Worker starts and subscribes	"Subscribed to flows:live"
6	Worker loads all 3 models	Status shows all True
7	Worker processes flows	flows_scored incrementing
8	Worker inference < 200ms	avg_inference_ms in stats
9	Worker detects anomalies	anomalies_detected > 0
10	Worker publishes alerts	Messages on alerts:live
11	Worker publishes scores	Messages on ml:scored
12	AlertEngine subscribes	"Subscribed to alerts:live"
13	AlertEngine inserts alerts	New rows in alerts table
14	Alert has ml_scores JSONB	composite, if, ae, rf
15	Alert has flow_ids array	Reference to source flow
16	FlowScoreUpdater subscribes	"Subscribed to ml:scored"
17	Flows updated with scores	anomaly_score column set
18	is_anomaly set correctly	true for anomalous flows
19	LLM Gateway imports	No errors
20	4 prompt templates load	alert_analysis, briefing, ip, translation
21	Task routing correct	DeepSeek→analysis, Groq→realtime
22	POST /ml/predict works	Returns composite score
23	ml-worker container stable	"Up" not "Restarting"
24	docker compose ps	All 5 services running
25	Retrain saves preprocessor artifacts	2 new .pkl files
26	Pipeline E2E: capture → score → alert	New alert appears in DB
27	Query anomalous flows	`WHERE is_anomaly = true` returns results
28	Query ML-generated alerts	`SELECT * FROM alerts WHERE category IN ('ddos','port_scan')`

Scope Adherence Verification

Requirement	Source	Verification
ML Worker subscribes flows:live	PART4 §8.1	Redis pubsub subscribe
Inference < 200ms per flow	PART4 §8.2	avg_inference_ms stat
Models pre-loaded in memory	PART4 §8.3	ModelManager.load_all() at startup
Alert categories from RF	PART4 §5.3	dos→ddos, probe→port_scan, etc
Alert severity from composite	PART4 §1.2	≥0.90=critical, ≥0.75=high...
Alert table schema	PART2 §4.2	All columns per spec
alerts:live Redis channel	PART2 §5.2	WebSocket event: new_alert
LLM provider routing	PART4 §9.1	DeepSeek, Groq, GLM mapping
LLM prompt templates	PART4 §9.2	4 templates per spec
POST /ml/predict	PART2 §5.1	Endpoint implemented

Blockers

Blocker	Severity	Mitigation
Preprocessor artifacts missing	Critical	Retrain with updated train_all.py first
No LLM API keys yet	Expected	Gateway scaffolded, full impl Week 4
Flow ID may not match DB	Medium	Handle gracefully if flow_id not found
TensorFlow model loading slow		~2-3 seconds at startup, then in-memory

Tomorrow's Preview (Day 12)

- Run hyperparameter tuning on VPS (`tune_models.py`)
- LLM API endpoints: POST /llm/chat (streaming), POST /llm/analyze-alert/{id}
- Begin Threat Intel integration: OTX + AbuseIPDB service
- WebSocket enhancement: broadcast alerts:live to connected browsers

Reference Documents

Document	Section	Purpose
MASTER_DOC_PART4	§8.1-8.3	ML Worker inference pipeline
MASTER_DOC_PART4	§1.2	Ensemble scoring + alert thresholds
MASTER_DOC_PART4	§5.3	RF label → alert category mapping
MASTER_DOC_PART4	§9.1-9.3	LLM Gateway architecture + prompts
MASTER_DOC_PART2	§4.2	Alerts table schema

Document	Section	Purpose
MASTER_DOC_PART2	§5.1	ML + LLM API endpoints
MASTER_DOC_PART2	§5.2	WebSocket events (alerts:live)
MASTER_DOC_PART2	§6.1	Redis pub/sub architecture

Task workflow for Day 11 (Week 3 Day 2) — ThreatMatrix AI Sprint 3
Focus: ML Worker + Alert Engine + Flow Score Updater + LLM Gateway Scaffold
Owner: Lead Architect — Frontend deferred to Full-Stack Dev